T1. Introducción

Historia

La informática gráfica ha evolucionado a lo largo de varias décadas, comenzando en los años 50 con el desarrollo de la **Whirlwind** (1951), que introdujo los **tubos de rayos catódicos** (TRC) como salida gráfica. Durante los años 50 y 60, se crearon los primeros videojuegos como **"Tennis for Two"** (1958) y se desarrollaron tecnologías clave como los **CAD/CAM** y el **DVST** para gráficos estáticos. En 1966, **Ivan Sutherland** presentó el primer **casco de visión estereoscópica**, precursor de la realidad virtual.

En los años 70, los **ordenadores personales** se popularizaron. **Magnavox Odyssey** (1972) fue la primera consola de videojuegos y **"Pong"** (1972) el primer gran éxito en el ámbito del entretenimiento. En esta década, se desarrollaron tecnologías clave como el **suavizado de superficies** y la **texturización 3D**.

Durante los años 80, la **industria de los gráficos** dio un gran salto con el uso de **tarjetas gráficas**, el desarrollo de **gráficos bitmap** y la creación de efectos visuales por ordenador en cine, como en **"Tron"** (1982).

En los años 90, **OpenGL** se convirtió en el estándar para gráficos 3D, mientras que **Microsoft** desarrolló **Direct3D**. En esta década, la industria de los gráficos se consolidó con avances como el **trazado de rayos** y las primeras **tarjetas gráficas programables**.

A principios de los 2000, OpenGL y Direct3D evolucionaron hacia arquitecturas gráficas más **programables**, permitiendo efectos visuales más complejos. **Vulkan**, lanzado en 2016, representó una nueva generación de API de bajo nivel, diseñada para ofrecer un mayor control sobre el hardware, mejorando el rendimiento y la eficiencia.

Hoy en día, las **APIs gráficas** como **Vulkan**, **Direct3D 12**, **Metal** y **WebGPU** permiten el desarrollo de aplicaciones gráficas avanzadas, como videojuegos y aplicaciones de **realidad virtual** y **aumentada**. Estas tecnologías han transformado áreas como el entretenimiento, la ciencia, la ingeniería y más, manteniendo la informática gráfica como un pilar fundamental en el avance tecnológico.

Visión. Interactividad

El **ojo humano** percibe el **movimiento** de manera continua captando numerosas imágenes estáticas que el cerebro usa para dar sensación de fluidez.

- **Persistencia**: fenómeno por el cual una imagen permanece durante un breve periodo después de que la fuente de luz haya desaparecido. Permite ver secuencias de imágenes como un movimiento continuo.

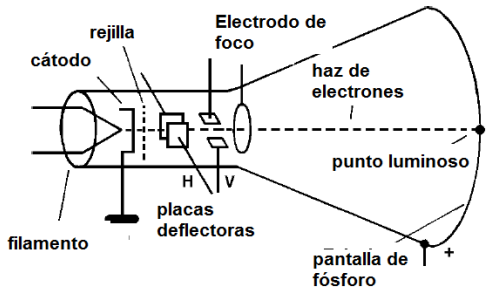
- **Motion Blur (desenfoque de movimiento)**: efecto de desenfoque que ocurre cuando un objeto en movimiento rápido deja una estela en la imagen. En el cine se usa para hacer el movimiento más realista.

Las imágenes en movimiento que vemos se forman con una serie de **imágenes fijas** mostradas en **rápida sucesión**.

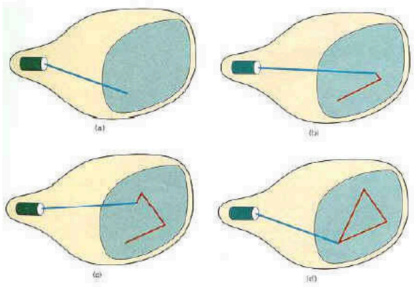
- **Frecuencia de refresco**: cuántas veces por segundo se actualiza una imagen en una pantalla. Para simular la visión humana se necesitan $24 - 30\ fps$.

Hardware Gráfico (TRC)

El **tubo de rayos catódicos (TRC)** consiste en un haz de e^- (haz catódico) emitido por un cañón de e^- que pasa a través de sistemas de enfoque y reflexión que lo redirige hacia posiciones específicas de una pantalla revestida de fósforo. El fósforo emite luz en la posición alcanzada por los e^-.



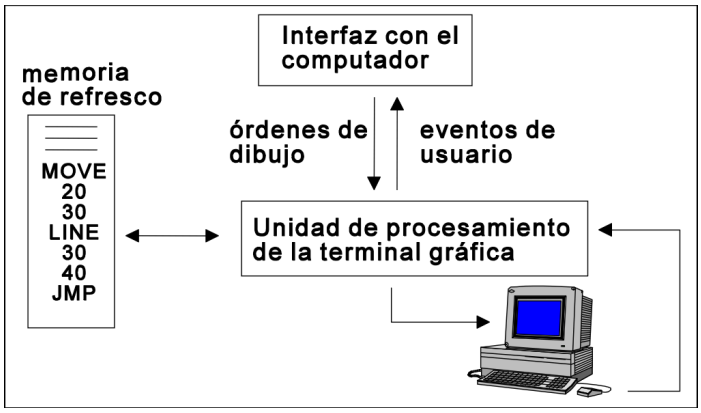
- <u>Terminales vectoriales</u>



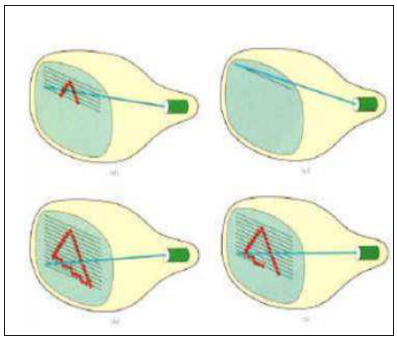
Las **terminales vectoriales** son tubos de rayos catódicos en el que el haz de e^- recorre aleatoriamente la pantalla de acuerdo con unas órdenes de dibujo.

- **Unidad de Procesamiento de la Terminal Gráfica**: procesa las órdenes de dibujo y las traduce en comandos para la pantalla. Accede y actualiza la memoria de refresco para mantener la imagen visible.

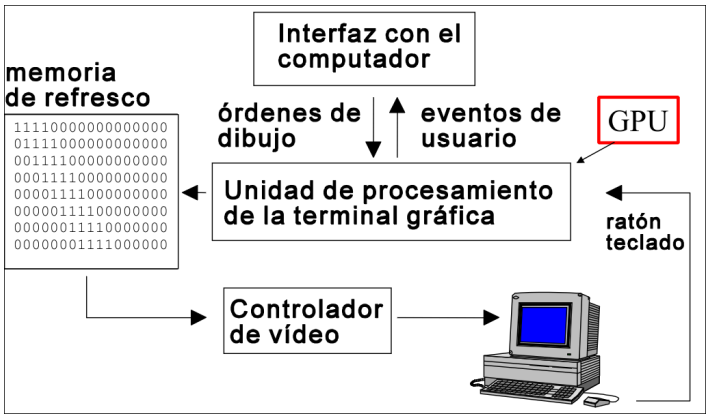
- **Memoria de refresco**: almacena las instrucciones necesarias para redibujar la imagen en la pantalla constantemente para que no desaparezca.



- <u>Terminales Ráster</u>



Las **terminales ráster** son TRC en el que el haz de e^- recorre la pantalla por líneas (625 líneas), desde la parte superior izquierda a la inferior derecha. Los datos mostrados por pantalla se extrapolan a una matriz **bidimensional** de datos que son los que se muestran en la pantalla. Cada entrada es un **Píxel, intersección** entre **fila** y **columna**.



El conjunto de valores de los píxeles se almacena en una zona de memoria denominada **Frame-Buffer**. El dispositivo o aplicación varía sus valores y estos se vuelcan (swap) a la pantalla en un momento dado.

Para la profundidad de color (bpp) es el número de **bits por pixel**, y determina la cantidad de colores que se pueden mostrar por pantalla:

$bpp \leq 8 \Rightarrow$ los valores de los píxeles son tonos RGB.

$bpp > 8 \Rightarrow$ se usa el modelo RGB estándar (16 bit de color).

Para obtener el **color real** se necesitan 24 bits por pixel (8 para cada uno). Para ahorrar memoria, en profundidades de color mayores a 8, se puede usar una **paleta** o GLUT con una entrada para cada posible valor de un píxel en una imagen, permitiendo **indexar** los colores usando el mínimo número de bits posible.

Otros tipos de terminales ráster con distintas bpp, frecuencia de refresco y resolución horizontal o vertical son: pantallas de plasma, LCD, LED, OLED...

En el caso de los LED, los computadores rápidamente **abandonaron** las **625 líneas** ya que estas estaban limitadas por la capacidad de **emisión**. Al poder procesar más píxeles y aumentar la velocidad de refresco se disminuye el cansancio ocular del usuario.

Visión 3D

La **visión en 3D** se basa en un principio fundamental: cada ojo percibe una imagen ligeramente distinta del mismo objeto.

Una de las técnicas más conocidas para lograr este efecto es la del **anaglifo**. En este método, se **superponen** dos **imágenes** (una correspondiente a cada ojo) utilizando distintos **colores**. Al usar gafas con filtros de esos mismos colores, cada ojo percibe únicamente la imagen que le corresponde, permitiendo al cerebro reconstruir la sensación de profundidad.

Otra forma es la **visión activa**, se utilizan gafas especiales que alternan rápidamente la imagen que ve cada ojo. La pantalla muestra de forma **secuencial** la imagen del ojo derecho y la del izquierdo, **sincronizándose** con las gafas, que bloquean un ojo mientras el otro recibe la imagen correspondiente. Este proceso ocurre tan rápido que el cerebro lo percibe como una imagen continua en tres dimensiones. Por otro lado, la **visión en 3D pasiva** utiliza un enfoque distinto. Aquí, ambas imágenes (la del ojo derecho y la del izquierdo) se muestran **simultáneamente** en la pantalla, pero con **diferentes polarizaciones** de la luz. Las gafas pasivas filtran estas polarizaciones, de modo que cada ojo solo percibe su imagen correspondiente.

Un caso especial en la evolución de estas tecnologías son los dispositivos de tipo **HMD** (Head-Mounted Display), desarrollados desde la década de 1960. Estos dispositivos colocan pequeñas pantallas frente a cada ojo, mostrando imágenes específicas para cada uno, lo que permite una experiencia inmersiva en tres dimensiones.

Conceptos básicos

- **Ráster**: representación de imágenes como una matriz de píxeles.
- **Pixel** (picture element): cada uno de los puntos de color que se muestran en una pantalla.
- **Gráficos vectoriales**: formato de imagen compuesto por objetos geométricos (puntos, líneas, polígonos), cuyos atributos matemáticos se almacenan. Permiten modificar el tamaño de una imagen sin pérdidas de calidad.

- **Renderizado**: proceso de creación de una imagen a partir de un modelo, calculando el color de cada píxel en la imagen según las propiedades del modelo.

- <u>Double buffer</u>

Se usan **dos frame buffer**:

- Un **buffer primario** cuyo contenido se muestra por pantalla.
- Un **buffer secundario** o trasero en el que se realiza el **renderizado** de las imágenes. Cuando se termina de renderizar en el buffer secundario, se **copia** su contenido al **principal**. Así se evita que el espectador pueda ver el cambio entre frames si no se renderiza lo suficientemente rápido.

Se **dibujan** en el buffer **secundario** los objetos en el **orden** en el que llegan. Una vez lista la imagen, se realiza un **swap**, mostrando instantáneamente el nuevo frame en el buffer primario. Se **limpia** el buffer secundario y se repite.

- <u>Depth Buffer</u>

El **Z-Buffer** o buffer de profundidad se usa para **gestionar** la **profundidad** de los objetos y resolver cuáles (o que parte de ellos) son **visibles** y cuáles están **detrás** de otros. Tiene el mismo ancho y alto que el frame-buffer y una profundidad de 16, 24, o 32 bits.

Cada píxel tiene una coordenada X e Y, y un valor de profundidad (Z) que indica su **distancia** con respecto a la **cámara**, de manera que cuando se renderiza (escritura en frame buffer y z-buffer) un nuevo pixel se **compara** su valor con el del anterior. En función del resultado, lo **reemplazará** (si es inferior se dibuja y actualiza el z-buffer) o será **descartado**.

Nota: Se puede implementar tanto en hardware como en software, y no se mantiene el valor inicial del Z-Buffer.

El **algoritmo del pintor** consiste en dibujar los objetos de **atrás hacia adelante**, ordenándolos según su coordenada Z (de mayor a menor). Si dos objetos se intersecan, no sabe determinar cuál debe dibujarse primero.

Pipeline

El **pipeline gráfico** es un modelo conceptual que describe los **pasos** que un sistema gráfico debe dar para **renderizar** una escena 3D en una pantalla 2D.

- <u>Aplicación</u>

La geometría se construye a partir de vértices, que constituyen los elementos primarios de todos los objetos. Cada **vértice** representa un **punto en el espacio**

3D y contiene diversos **atributos** asociados que definen sus características visuales. La **combinación** de **múltiples vértices** organizados según patrones específicos forma las primitivas básicas como triángulos, líneas o puntos, que a su vez se ensamblan para crear objetos complejos.

- <u>Vertex shader</u>

Una vez definida la geometría en coordenadas locales, el programador debe **transferir** estos **datos** a la GPU mediante **Vertex Array Objects** (VAO) y **Vertex Buffer Objects** (VBOs). Ambos son contenedores de memoria en la GPU que almacenan eficientemente los datos de vértices. Su creación y gestión representan una responsabilidad del desarrollador, ya que determinan cómo se almacenan y acceden los datos geométricos.

Un VAO encapsula la configuración completa de cómo OpenGL debe interpretar los **datos almacenados** en los buffers de vértices. Esta abstracción permite gestionar eficientemente **múltiples objetos** con diferentes configuraciones de atributos. La salida del Vertex Shader debe incluir necesariamente la **posición** del **vértice** transformada al **espacio global** de la escena.

- <u>Ensembled</u>

Una vez que el Vertex Shader ha procesado todos los vértices y los ha transformado al espacio global de la escena, el pipeline procede con la etapa de **ensamblado de primitivas**. En esta fase, los **vértices individuales** se **organizan** y **agrupan** según la topología especificada por el programador para formar las primitivas geométricas básicas que constituyen los objetos de la escena.

- <u>Clipping</u>

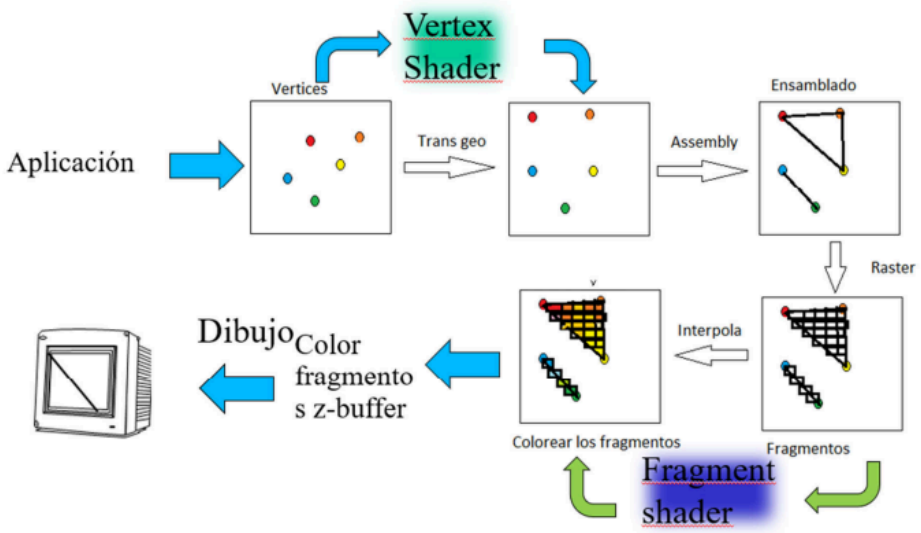
Es un proceso fundamental que garantiza que **solo** la **geometría visible** dentro del volumen de visión sea **procesada** . El clipping se realiza **contra el volumen** de vista definido por el frustum de la cámara, que comprende seis planos : near, far, left, right, top y bottom. Elimina primitivas que se encuentran completamente fuera del área visible, evitando así el costoso procesamiento de geometría que no contribuiría a la imagen final.

- <u>Rasterización</u>

Durante este proceso, se genera un **fragmento por cada píxel** que interseca con la primitiva, y para cada uno de estos fragmentos se **interpolan** de manera perspectivacorrecta todos los atributos de los vértices originales. Esta interpolación es esencial para atributos como **colores**, coordenadas de **textura**, **normales** y valores de **profundidad**, asegurando que las transiciones entre vértices sean suaves y visualmente coherentes.

- <u>Fragment shader</u>

La principal responsabilidad del Fragment Shader es **calcular** el **color final** del fragmento mediante la implementación de algoritmos de **sombreado** que simulan el comportamiento de la luz y los materiales. Esto incluye la aplicación de **texturas** mediante **muestreo**, el cálculo de modelos de iluminación.

- <u>Z-buffer y Blending</u>

La prueba de profundidad (Z-test) **compara** el **valor Z** del fragmento actual con el **almacenado** en el **Z-buffer**, descartando aquellos fragmentos que se encuentran ocultos por geometría más cercana a la cámara. Por el contrario, el **Blending mezcla** los **colores** de los fragmentos. Se crea el **Color Buffer.**